

OOP Pazymiu Sistema

1.5

Sugeneruota Doxygen 1.17.0

skyrius 1

OOP Pazymiu Sistema

Studentu pazymiu skaiciavimo sistema, realizuota C++ kalba.

Programa leidzia:

- iversti studentu duomenis terminale;
- nuskaityti studentus is failo;
- sugeneruoti studentu failus;
- skaiciuoti galutini bala pagal vidurki arba mediana;
- rusiuoti studentus;
- skirstyti studentus i vargsiukus ir kietiokus;
- testuoti klases metodus ir papildomus unit testus.

1.1 Naudojamos technologijos

- C++20
- CMake
- Catch2 unit testams
- Doxygen dokumentacijai
- Graphviz Doxygen grafikams
- MiKTeX arba Overleaf PDF dokumentacijai

1.2 Projekto struktura

```
.
CMakeLists.txt
Doxyfile
README.md
vector/
    funkcijos.h
    funkcijos.cpp
    main.cpp
    tests.cpp
tests/
    unit_tests.cpp
```

1.3 Release'ai

1.3.1 v1.0

Pradine programos versija:

- studentu duomenų įvedimas;
- galutinio balo skaičiavimas;
- failų nuskaitymas;
- studentų išvedimas.

1.3.2 v1.1

Pridetas studentų skirstymas į dvi grupes:

- vargsiukai;
- kietiakai.

Testuotos skirtingos konteinerių strategijos.

1.3.3 v1.2

Pridetas darbas su skirtingais STL konteineriais:

- `std::vector`;
- `std::list`;
- `std::deque`.

Buvo lyginamos konteinerių veikimo strategijos ir skirstymo į grupes laikas.

1.3.4 v1.5

Galutinė objektinio programavimo versija:

- sukurta abstrakti bazinė klasė [Zmogus](#);
- sukurta išvestinė klasė [Studentas](#);
- realizuota Rule of Five:
 - destruktorius;
 - copy konstruktorius;
 - copy assignment operatorius;
 - move konstruktorius;
 - move assignment operatorius;
- pridėti Catch2 unit testai;
- prideta Doxygen dokumentacija;
- paruoštas CMakeLists.txt.

1.4 Idiegimo instrukcija Windows aplinkoje

1.4.1 Reikalinga programine iranga

Windows aplinkoje reikia tureti:

- C++20 palaikanti kompiliatoriu, pvz. MSVC arba MinGW;
- CMake;
- Git;
- Doxygen;
- Graphviz, jei norima matyti Doxygen klasiu grafus;
- MiKTeX, jei norima PDF dokumentacija generuoti lokaliai.

Catch2 papildomai diegti nereikia, nes jis automatiskai atsiunciamas per CMake FetchContent.

1.4.2 CMake diegimas

Atsisiuskite CMake Windows installer ir diegimo metu pasirinkite:

```
Add CMake to the system PATH for all users
```

arba:

```
Add CMake to the PATH for current user
```

Po diegimo uždarykite terminalą ir atidarykite jį iš naujo.

Patikrinimas:

```
cmake --version
```

Jeigu komanda neatpažistama, reikia į Windows PATH pridėti:

```
C:\Program Files\CMake\bin
```

1.4.3 Doxygen ir Graphviz

Doxygen reikalingas HTML ir LaTeX dokumentacijos generavimui.

Graphviz reikalingas klasių grafikams.

Patikrinimas:

```
doxygen --version  
dot -V
```

Jeigu dot komanda neatpažistama, į Windows PATH reikia pridėti Graphviz bin katalogą, pvz.:

```
C:\Program Files\Graphviz\bin
```

Jeigu Doxygen vis tiek neranda Graphviz, Doxyfile faile galima nurodyti:

```
DOT_PATH = "C:/Program Files/Graphviz/bin"
```

1.5 Kompiliavimas su CMake Windows aplinkoje

Is pagrindinio projekto katalogo paleiskite:

```
cmake -S . -B build  
cmake --build build
```

Programos paleidimas:

```
build\Debug\programa.exe
```

Jeigu naudojama Release konfigūracija:

```
cmake -S . -B build  
cmake --build build --config Release  
build\Release\programa.exe
```

1.6 Unit testai

Unit testams naudojamas Catch2 framework'as.

Testai yra faile:

```
tests/unit_tests.cpp
```

Testai tikrina:

- abstrakčios bazinės klasės [Zmogus](#) veikima;
- [Studentas](#) numatytąjį konstruktoriu;
- [Studentas](#) parametrinį konstruktoriu;
- Rule of Five metodus;
- galutinio balo skaičiavimą;
- failo nuskaitymą;
- studentų skirstymą naudojant `std::vector`.

Testų paleidimas Windows aplinkoje:

```
cmake -S . -B build  
cmake --build build  
ctest --test-dir build --output-on-failure
```

Jeigu naudojama Release konfigūracija:

```
ctest --test-dir build -C Release --output-on-failure
```

1.7 Doxygen dokumentacija

Dokumentacija generuojama naudojant Doxyfile.

HTML ir LaTeX dokumentacijos generavimas:

```
doxygen Doxyfile
```

Po generavimo turi atsirasti:

```
docs/html/  
docs/latex/
```

HTML dokumentacijos pradžios failas:

```
docs/html/index.html
```

1.8 PDF dokumentacijos generavimas Windows aplinkoje

Jeigu idiegtas MiKTeX, PDF galima sugeneruoti taip:

```
cd docs\latex  
pdflatex refman.tex  
makeindex refman.idx  
pdflatex refman.tex  
pdflatex refman.tex
```

Po kompiliavimo turi atsirasti:

```
docs\latex\refman.pdf
```

Jeigu MiKTeX arba kito LaTeX kompiliatoriaus nėra, galima naudoti Overleaf:

1. Sugeneruokite dokumentaciją su komanda `doxygen Doxyfile`.
2. Ikelkite visą `docs/latex` katalogo turinį į Overleaf.
3. Overleaf projekte pagrindinių failų pasirinkite `refman.tex`.
4. Sugeneruokite PDF.
5. Atsisiusta `refman.pdf` idėkite atgal į projekto katalogą:
`docs\latex\refman.pdf`

1.9 Doxygen grafikai

Doxygen klasiu grafams reikalingas Graphviz.

Svarbiausi Doxyfile nustatymai grafikams:

```
HAVE_DOT           = YES
CLASS_DIAGRAMS     = YES
CLASS_GRAPH        = YES
COLLABORATION_GRAPH = YES
INCLUDE_GRAPH      = YES
INCLUDED_BY_GRAPH  = YES
GRAPHICAL_HIERARCHY = YES
DIRECTORY_GRAPH    = YES
```

Jeigu collaboration graph nerodomas, pirmiausia patikrinkite:

```
dot -V
```

Taip pat istrinkite sena dokumentacija ir sugeneruokite is naujo:

```
rmdir /s /q docs
doxygen Doxyfile
```

1.10 Naudojimosi instrukcija

Paleidus programa vartotojui pateikiamas meniu. Jame galima pasirinkti:

- duomenų įvedimo ranka;
- duomenų nuskaitymas iš failo;
- studentų failo generavimas;
- rusiavimas;
- skirstymas į grupes;
- konteinerių strategijų testavimas;
- klasių metodų testavimas.

Bendras veikimo principas:

1. Pasirenkamas duomenų gavimo būdas.
2. Studentų duomenys apdorojami.
3. Apskaiciuojami galutiniai balai.
4. Studentai gali būti surusiuoti arba suskirstyti.
5. Rezultatai išvedami į terminalą arba failus.

1.11 Rezultatu komentaras

Naudojant STL konteinerius pastebima, kad `std::vector` dažnai yra efektyvus dėl nuoseklaus atminties išdėstymo. Tačiau salinimo operacijose `std::list` arba `std::deque` kai kuriais atvejais gali veikti patogiau, priklausomai nuo pasirinktos skirstymo strategijos.

Unit testai leidžia greitai patikrinti, ar pakeitus kodą nebuvo sugadintas pagrindinis funkcionalumas. Ypač svarbu testuoti Rule of Five metodus, nes jie susiję su objektu kopijavimu, perkėlimu ir resursų valdymu.

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

tests	??
unit_tests.cpp	??
vector	??
funkcijos.cpp	??
funkcijos.h	??
main.cpp	??

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	??
Studentas	??

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas

Studentas su namu darbu, egzamino ir galutinio balo duomenimis ??

Zmogus

Abstrakti bazinė klase bendriems žmogaus duomenims saugoti ??

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

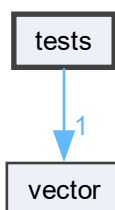
unit_tests.cpp	..	??
funkcijos.cpp	..	??
funkcijos.h		
Studentu pazymiu sistemos klasiu ir funkciju deklaracijos	..	??
main.cpp	..	??

skyrius 6

Direktorių dokumentacija

6.1 tests Direktorijos aprašas

Directory dependency graph for tests:



Failai

- failas [unit_tests.cpp](#)

6.2 vector Direktorijos aprašas

Failai

- failas [funkcijos.cpp](#)
- failas [funkcijos.h](#)
 - Studentu pazymiu sistemos klasiu ir funkciju deklaracijos.*
- failas [main.cpp](#)

skyrius 7

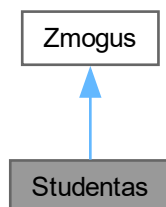
Klasės Dokumentacija

7.1 Studentas Klasė

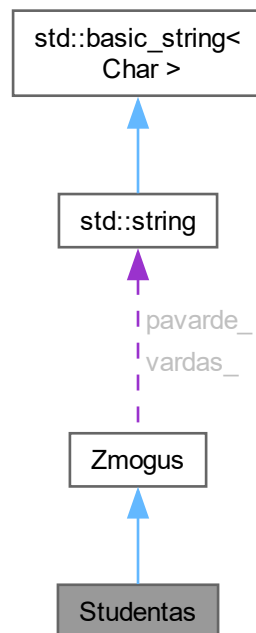
Studentas su namu darbu, egzamino ir galutinio balo duomenimis.

```
#include <funkcijos.h>
```

Paveldimumo diagrama Studentas:



Bendradarbiavimo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
Sukuria tuscia studenta.
- `~Studentas ()`
Sunaikina studento objekta.
- `Studentas (const std::string &vardas, const std::string &pavarde, uint8_t egzaminas, uint16_t namuDarbaiVid100, uint16_t namuDarbaiMed100, const std::vector< short int > &ND={})`
Sukuria studenta su perduotais duomenimis.
- `Studentas (const Studentas &other)`
Copy konstruktorius.
- `Studentas & operator= (const Studentas &other)`
Copy priskyrimo operatorius.
- `Studentas (Studentas &&other) noexcept`
Move konstruktorius.
- `Studentas & operator= (Studentas &&other) noexcept`
Move priskyrimo operatorius.
- `uint16_t getNamuDarbaiVid100 () const`
- `uint16_t getNamuDarbaiMed100 () const`
- `uint8_t getEgzaminas () const`
- `const std::vector< short int > & getND () const`
- `void setNamuDarbaiVid100 (uint16_t value)`
*Nustato ND vidurki * 100.*
- `void setNamuDarbaiMed100 (uint16_t value)`

- *Nustato ND mediana * 100.*
- void `setEgzaminas` (uint8_t value)
Nustato egzamino pazymi.
- void `setND` (const std::vector< short int > &value)
Pakeicia visa ND pazymių vektoriu.
- void `addND` (short int pazymys)
Prideda viena ND pazymi.
- double `galutinisVid` () const
- double `galutinisMed` () const
- std::string `tipas` () const override

Vieši Metodai inherited from `Zmogus`

- `Zmogus` ()
Sukuria tuscia zmogu.
- `Zmogus` (const std::string &vardas, const std::string &pavarde)
Sukuria zmogu su vardu ir pavarde.
- virtual `~Zmogus` ()=default
Virtualus destruktorius saugiam paveldetu objektu naikinimui.
- const std::string & `getVardas` () const
- const std::string & `getPavarde` () const
- void `setVardas` (const std::string &value)
Nustato varda.
- void `setPavarde` (const std::string &value)
Nustato pavarde.

Additional Inherited Members

Apsaugoti Atributai inherited from `Zmogus`

- std::string `vardas_`
Zmogaus vardas.
- std::string `pavarde_`
Zmogaus pavarde.

7.1.1 Smulkus aprašymas

`Studentas` su namu darbu, egzamino ir galutinio balo duomenimis.

Klase realizuoja Rule of Five: destruktoriu, copy konstruktoriu, copy priskyrimo operatoriu, move konstruktoriu ir move priskyrimo operatoriu.

Apibrėžimas failo `funkcijos.h` eilutėje 74.

7.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.1.2.1 Studentas() [1/4]

```
Studentas::Studentas ()
```

Sukuria tuscia studenta.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 27.

7.1.2.2 ~Studentas()

```
Studentas::~~Studentas () [default]
```

Sunaikina studento objekta.

7.1.2.3 Studentas() [2/4]

```
Studentas::Studentas (  
    const std::string & vardas,  
    const std::string & pavarde,  
    uint8_t egzaminas,  
    uint16_t namuDarbaiVid100,  
    uint16_t namuDarbaiMed100,  
    const std::vector< short int > & ND = {})
```

Sukuria studenta su perduotais duomenimis.

Parametrai

<i>vardas</i>	Studento vardas.
<i>pavarde</i>	Studento pavarde.
<i>egzaminas</i>	Egzamino pazymys.
<i>namuDarbaiVid100</i>	ND vidurkis, padaugintas is 100.
<i>namuDarbaiMed100</i>	ND mediana, padauginta is 100.
<i>ND</i>	Namu darbu pazymiai.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 34.

7.1.2.4 Studentas() [3/4]

```
Studentas::Studentas (  
    const Studentas & other)
```

Copy konstruktorius.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 48.

7.1.2.5 Studentas() [4/4]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Move konstruktorius.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 67.

7.1.3 Metodų Dokumentacija

7.1.3.1 addND()

```
void Studentas::addND (  
    short int pazymys)
```

Prideda viena ND pazymi.

Parametrai

<i>pazymys</i>	Pazymys.
----------------	----------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 127.

7.1.3.2 galutinisMed()

```
double Studentas::galutinisMed () const
```

Gražina

Galutinis balas pagal ND mediana.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 135.

7.1.3.3 galutinisVid()

```
double Studentas::galutinisVid () const
```

Gražina

Galutinis balas pagal ND vidurki.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 131.

7.1.3.4 getEgzaminas()

```
uint8_t Studentas::getEgzaminas () const
```

Gražina

Egzamino pazymys.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 103.

7.1.3.5 getNamuDarbaiMed100()

```
uint16_t Studentas::getNamuDarbaiMed100 () const
```

Gražina

ND mediana, padauginta is 100.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 99.

7.1.3.6 getNamuDarbaiVid100()

```
uint16_t Studentas::getNamuDarbaiVid100 () const
```

Gražina

ND vidurkis, padaugintas is 100.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 95.

7.1.3.7 getND()

```
const std::vector< short int > & Studentas::getND () const
```

Gražina

Konstantine nuoroda i ND pazymių vektorių.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 107.

7.1.3.8 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other)
```

Copy priskyrimo operatorius.

Parametrai

<i>other</i>	Kopijuojamas objektas.
--------------	------------------------

Gražina

Si objektas.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 55.

7.1.3.9 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other) [noexcept]
```

Move priskyrimo operatorius.

Parametrai

<i>other</i>	Perkeliamas objektas.
--------------	-----------------------

Gražina

Si objektas.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 78.

7.1.3.10 setEgzaminas()

```
void Studentas::setEgzaminas (
    uint8_t value)
```

Nustato egzamino pazymi.

Parametrai

<i>value</i>	Nauja reiksme.
--------------	----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 119.

7.1.3.11 setNamuDarbaiMed100()

```
void Studentas::setNamuDarbaiMed100 (
    uint16_t value)
```

Nustato ND mediana * 100.

Parametrai

<i>value</i>	Nauja reiksme.
--------------	----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 115.

7.1.3.12 setNamuDarbaiVid100()

```
void Studentas::setNamuDarbaiVid100 (
    uint16_t value)
```

Nustato ND vidurki * 100.

Parametrai

<i>value</i>	Nauja reiksme.
--------------	----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 111.

7.1.3.13 setND()

```
void Studentas::setND (
    const std::vector< short int > & value)
```

Pakeicia visa ND pazymiu vektoriu.

Parametrai

<i>value</i>	Nauji pazymiai.
--------------	-----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 123.

7.1.3.14 tipas()

```
std::string Studentas::tipas () const [override], [virtual]
```

Gražina

Tekstas "Studentas".

Realizuoja [Zmogus](#).

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 139.

Dokumentacija šiai klasei sugeneruota iš šių failų:

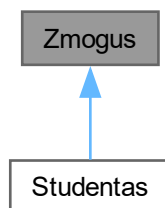
- [funkcijos.h](#)
- [funkcijos.cpp](#)

7.2 Zmogus Klasė

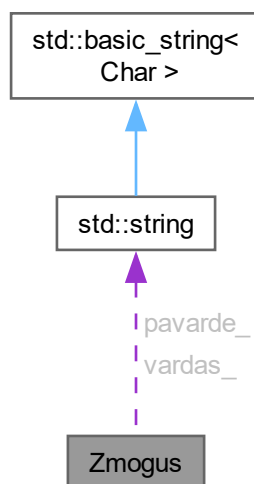
Abstrakti bazine klase bendriems zmogaus duomenims saugoti.

```
#include <funkcijos.h>
```

Paveldimumo diagrama Zmogus:



Bendradarbiavimo diagrama Zmogus:



Vieši Metodai

- **Zmogus** ()
Sukuria tuscia zmogu.
- **Zmogus** (const std::string &vardas, const std::string &pavarde)
Sukuria zmogu su vardu ir pavarde.

- virtual `~Zmogus()` = default
Virtualus destruktorius saugiam paveldetu objektu naikinimui.
- const std::string & `getVardas()` const
- const std::string & `getPavarde()` const
- void `setVardas` (const std::string &value)
Nustato varda.
- void `setPavarde` (const std::string &value)
Nustato pavarde.
- virtual std::string `tipas()` const = 0

Apsaugoti Atributai

- std::string `vardas_`
Zmogaus vardas.
- std::string `pavarde_`
Zmogaus pavarde.

7.2.1 Smulkus aprašymas

Abstrakti bazine klase bendriems zmogaus duomenims saugoti.

Klase saugo varda ir pavarde. Pure virtual metodas `tipas()` padaro klase abstrakcia, todėl tiesioginiu `Zmogus` objektu kurti negalima.

Apibrėžimas failo `funkcijos.h` eilutėje 33.

7.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus ()
```

Sukuria tuscia zmogu.

Apibrėžimas failo `funkcijos.cpp` eilutėje 4.

7.2.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde)
```

Sukuria zmogu su vardu ir pavarde.

Parametrai

<code>vardas</code>	Vardas.
---------------------	---------

<i>pavarde</i>	Pavarde.
----------------	----------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 7.

7.2.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [virtual], [default]
```

Virtualus destruktorius saugiam paveldetu objektu naikinimui.

7.2.3 Metodų Dokumentacija

7.2.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde () const
```

Gražina

Zmogaus pavarde.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 14.

7.2.3.2 getVardas()

```
const std::string & Zmogus::getVardas () const
```

Gražina

Zmogaus vardas.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 10.

7.2.3.3 setPavarde()

```
void Zmogus::setPavarde (  
    const std::string & value)
```

Nustato pavarde.

Parametrai

<i>value</i>	Nauja pavarde.
--------------	----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 22.

7.2.3.4 setVardas()

```
void Zmogus::setVardas (
    const std::string & value)
```

Nustato vardą.

Parametrai

<i>value</i>	Naujas vardas.
--------------	----------------

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 18.

7.2.3.5 tipas()

```
virtual std::string Zmogus::tipas () const [pure virtual]
```

Gražina

Konkretus paveldetos klasės tipas.

Realizuota [Studentas](#).

7.2.4 Atributų Dokumentacija

7.2.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Zmogaus pavardė.

Apibrėžimas failo [funkcijos.h](#) eilutėje 36.

7.2.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Zmogaus vardas.

Apibrėžimas failo [funkcijos.h](#) eilutėje 35.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [funkcijos.h](#)
- [funkcijos.cpp](#)

skyrius 8

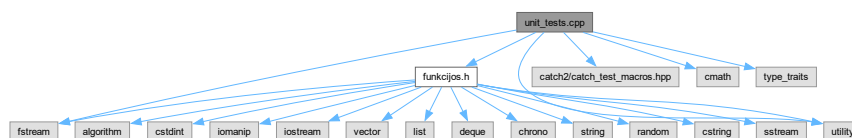
Failo Dokumentacija

8.1 README.md Failo Nuoroda

8.2 unit_tests.cpp Failo Nuoroda

```
#include "funkcijos.h"  
#include <catch2/catch_test_macros.hpp>  
#include <cmath>  
#include <fstream>  
#include <type_traits>  
#include <utility>
```

Įtraukimo priklausomybių diagrama unit_tests.cpp:



Funkcijos

- [TEST_CASE](#) ("Zmogus yra abstrakti bazine klase", "[zmogus][oop]")
- [TEST_CASE](#) ("Numatytasis konstruktorius sukuria tuscia studenta", "[studentas][constructor]")
- [TEST_CASE](#) ("Parametrinis konstruktorius uzpildo visus laukus", "[studentas][constructor]")
- [TEST_CASE](#) ("Rule of Five: copy konstruktorius atlieka gilia kopija", "[studentas][rule-of-five]")
- [TEST_CASE](#) ("Rule of Five: copy assignment teisingai priskiria reiksmes", "[studentas][rule-of-five]")
- [TEST_CASE](#) ("Rule of Five: move konstruktorius perkelia duomenis", "[studentas][rule-of-five]")
- [TEST_CASE](#) ("Rule of Five: move assignment perkelia duomenis", "[studentas][rule-of-five]")
- [TEST_CASE](#) ("Rule of Five: destruktorius leidzia saugiai naikinti per bazine rodykle", "[studentas][rule-of-five]")
- [TEST_CASE](#) ("Galutinio balo skaiciavimas pagal vidurki ir mediana", "[studentas][calculation]")
- [TEST_CASE](#) ("Failo nuskaitymas issaugo ND, kai saveND=true", "[file]")
- [TEST_CASE](#) ("Vector skirstymo strategija atskiria vargsiukus", "[split][vector]")

8.2.1 Funkcijos Dokumentacija

8.2.1.1 TEST_CASE() [1/11]

```
TEST_CASE (
    "Failo nuskaitymas issaugo ND)
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 120.

8.2.1.2 TEST_CASE() [2/11]

```
TEST_CASE (
    "Galutinio balo skaiciavimas pagal vidurki ir mediana" ,
    "" [studentas][calculation])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 113.

8.2.1.3 TEST_CASE() [3/11]

```
TEST_CASE (
    "Numatytasis konstruktorius sukuria tuscia studenta" ,
    "" [studentas][constructor])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 26.

8.2.1.4 TEST_CASE() [4/11]

```
TEST_CASE (
    "Parametrinis konstruktorius uzpildo visus laukus" ,
    "" [studentas][constructor])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 37.

8.2.1.5 TEST_CASE() [5/11]

```
TEST_CASE (
    "Rule of Five: copy assignment teisingai priskiria reiksmes" ,
    "" [studentas][rule-of-five])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 63.

8.2.1.6 TEST_CASE() [6/11]

```
TEST_CASE (
    "Rule of Five: copy konstruktorius atlieka gilia kopija" ,
    "" [studentas][rule-of-five])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 48.

8.2.1.7 TEST_CASE() [7/11]

```
TEST_CASE (
    "Rule of Five: destruktorius leidžia saugiai naikinti per bazinę rodyklę" ,
    "" [studentas][rule-of-five])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 107.

8.2.1.8 TEST_CASE() [8/11]

```
TEST_CASE (
    "Rule of Five: move assignment perkelia duomenis" ,
    "" [studentas][rule-of-five])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 92.

8.2.1.9 TEST_CASE() [9/11]

```
TEST_CASE (
    "Rule of Five: move konstruktorius perkelia duomenis" ,
    "" [studentas][rule-of-five])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 77.

8.2.1.10 TEST_CASE() [10/11]

```
TEST_CASE (
    "Vector skirstymo strategija atskiria vargsiukus" ,
    "" [split][vector])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 136.

8.2.1.11 TEST_CASE() [11/11]

```
TEST_CASE (
    "Zmogus yra abstrakti bazinė klase" ,
    "" [zmogus][oop])
```

Apibrėžimas failo [unit_tests.cpp](#) eilutėje 14.

8.3 unit_tests.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "funkcijos.h"
00002
00003 #include <catch2/catch_test_macros.hpp>
00004
00005 #include <cmath>
00006 #include <fstream>
00007 #include <type_traits>
00008 #include <utility>
00009
00010 namespace {
00011 constexpr double EPS = 1e-9;
00012 }
00013
00014 TEST_CASE("Zmogus yra abstrakti bazine klase", "[zmogus][oop]") {
00015     static_assert(std::is_abstract_v<Zmogus>);
00016     static_assert(std::is_base_of_v<Zmogus, Studentas>);
00017
00018     Studentas s("Jonas", "Jonaitis", 8, 900, 850, {8, 9, 10});
00019     Zmogus* z = &s;
00020
00021     REQUIRE(z->getVardas() == "Jonas");
00022     REQUIRE(z->getPavarde() == "Jonaitis");
00023     REQUIRE(z->tipas() == "Studentas");
00024 }
00025
00026 TEST_CASE("Numatytasis konstruktorius sukuria tuscia studenta", "[studentas][constructor]") {
00027     Studentas s;
00028
00029     REQUIRE(s.getVardas().empty());
00030     REQUIRE(s.getPavarde().empty());
00031     REQUIRE(s.getEgzaminas() == 0);
00032     REQUIRE(s.getNamuDarbaiVid100() == 0);
00033     REQUIRE(s.getNamuDarbaiMed100() == 0);
00034     REQUIRE(s.getND().empty());
00035 }
00036
00037 TEST_CASE("Parametrinis konstruktorius uzpildo visus laukus", "[studentas][constructor]") {
00038     Studentas s("Ona", "Onaite", 9, 875, 900, {8, 9, 10});
00039
00040     REQUIRE(s.getVardas() == "Ona");
00041     REQUIRE(s.getPavarde() == "Onaite");
00042     REQUIRE(s.getEgzaminas() == 9);
00043     REQUIRE(s.getNamuDarbaiVid100() == 875);
00044     REQUIRE(s.getNamuDarbaiMed100() == 900);
00045     REQUIRE((s.getND() == std::vector<short int>{8, 9, 10}));
00046 }
00047
00048 TEST_CASE("Rule of Five: copy konstruktorius atlieka gilia kopija", "[studentas][rule-of-five]") {
00049     Studentas original("Petras", "Petraitis", 7, 650, 700, {6, 7, 8});
00050     Studentas kopija(original);
00051
00052     original.setVardas("Pakeistas");
00053     original.setND({1, 2, 3});
00054
00055     REQUIRE(kopija.getVardas() == "Petras");
00056     REQUIRE(kopija.getPavarde() == "Petraitis");
00057     REQUIRE(kopija.getEgzaminas() == 7);
00058     REQUIRE(kopija.getNamuDarbaiVid100() == 650);
00059     REQUIRE(kopija.getNamuDarbaiMed100() == 700);
00060     REQUIRE((kopija.getND() == std::vector<short int>{6, 7, 8}));
00061 }
00062
00063 TEST_CASE("Rule of Five: copy assignment teisingai priskiria reiksmes", "[studentas][rule-of-five]") {
00064     Studentas a("A", "AA", 10, 950, 900, {9, 10});
00065     Studentas b("B", "BB", 4, 400, 400, {4});
00066
00067     b = a;
00068     a.setPavarde("Pakeista");
00069     a.addND(1);
00070
00071     REQUIRE(b.getVardas() == "A");
00072     REQUIRE(b.getPavarde() == "AA");
00073     REQUIRE(b.getEgzaminas() == 10);
00074     REQUIRE((b.getND() == std::vector<short int>{9, 10}));
00075 }
00076
00077 TEST_CASE("Rule of Five: move konstruktorius perkelia duomenis", "[studentas][rule-of-five]") {
00078     Studentas a("Ieva", "Ievaite", 8, 800, 850, {8, 8, 9});
00079     Studentas b(std::move(a));
00080
00081     REQUIRE(b.getVardas() == "Ieva");
00082     REQUIRE(b.getPavarde() == "Ievaite");

```

```

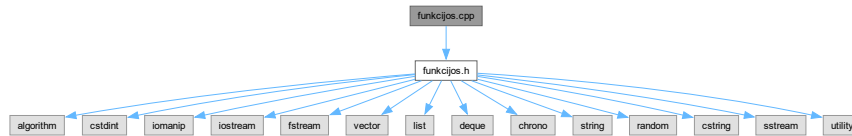
00083     REQUIRE(b.getEgzaminas() == 8);
00084     REQUIRE(b.getNamuDarbaiVid100() == 800);
00085     REQUIRE(b.getNamuDarbaiMed100() == 850);
00086     REQUIRE((b.getND() == std::vector<short int>{8, 8, 9}));
00087     REQUIRE(a.getEgzaminas() == 0);
00088     REQUIRE(a.getNamuDarbaiVid100() == 0);
00089     REQUIRE(a.getNamuDarbaiMed100() == 0);
00090 }
00091
00092 TEST_CASE("Rule of Five: move assignment perkelia duomenis", "[studentas][rule-of-five]") {
00093     Studentas a("Lukas", "Lukaitis", 9, 900, 900, {9, 9, 9});
00094     Studentas b;
00095
00096     b = std::move(a);
00097
00098     REQUIRE(b.getVardas() == "Lukas");
00099     REQUIRE(b.getPavarde() == "Lukaitis");
00100     REQUIRE(b.getEgzaminas() == 9);
00101     REQUIRE((b.getND() == std::vector<short int>{9, 9, 9}));
00102     REQUIRE(a.getEgzaminas() == 0);
00103     REQUIRE(a.getNamuDarbaiVid100() == 0);
00104     REQUIRE(a.getNamuDarbaiMed100() == 0);
00105 }
00106
00107 TEST_CASE("Rule of Five: destruktorius leidzia saugiai naikinti per bazine rodykle",
"[studentas][rule-of-five]") {
00108     Zmogus* z = new Studentas("Ruta", "Rutaitis", 10, 1000, 1000, {10});
00109     REQUIRE(z->tipas() == "Studentas");
00110     delete z;
00111 }
00112
00113 TEST_CASE("Galutinio balo skaiciavimas pagal vidurki ir mediana", "[studentas][calculation]") {
00114     Studentas s("Mantas", "Mantaitis", 8, 750, 850, {7, 8, 9});
00115
00116     REQUIRE(std::abs(s.galutinisVid() - 7.8) < EPS);
00117     REQUIRE(std::abs(s.galutinisMed() - 8.2) < EPS);
00118 }
00119
00120 TEST_CASE("Failo nuskaitymas issaugo ND, kai saveND=true", "[file]") {
00121     const std::string fileName = "test_input.txt";
00122     std::ofstream out(fileName);
00123     out << "Vardas Pavarde ND1 ND2 ND3 Egz.\n";
00124     out << "Tomas Tomaitis 5 6 7 8\n";
00125     out.close();
00126
00127     const auto studentai = readFile(fileName, true);
00128
00129     REQUIRE(studentai.size() == 1);
00130     REQUIRE(studentai[0].getVardas() == "Tomas");
00131     REQUIRE(studentai[0].getPavarde() == "Tomaitis");
00132     REQUIRE(studentai[0].getEgzaminas() == 8);
00133     REQUIRE((studentai[0].getND() == std::vector<short int>{5, 6, 7}));
00134 }
00135
00136 TEST_CASE("Vector skirstymo strategija atskiria vargsiukus", "[split][vector]") {
00137     std::vector<Studentas> studentai = {
00138         Studentas("A", "A", 4, 400, 400, {4}),
00139         Studentas("B", "B", 10, 1000, 1000, {10}),
00140         Studentas("C", "C", 5, 500, 500, {5})
00141     };
00142     std::vector<Studentas> vargsiukai;
00143
00144     splitVector3(studentai, vargsiukai);
00145
00146     REQUIRE(vargsiukai.size() == 1);
00147     REQUIRE(vargsiukai[0].getVardas() == "A");
00148     REQUIRE(studentai.size() == 2);
00149 }

```

8.4 funkcijos.cpp Failo Nuoroda

```
#include "funkcijos.h"
```

Įtraukimo priklausomybių diagrama funkcijos.cpp:



Funkcijos

- `std::istream & operator>> (std::istream &in, Studentas &s)`
Nuskaityti vieną studentą iš srauto.
- `std::ostream & operator<< (std::ostream &out, const Studentas &s)`
Išvesti vieną studentą į srautą.
- `std::vector< Studentas > readFile (const std::string &filename, bool saveND)`
Nuskaityti studentus iš failo.
- `std::vector< Studentas > readTerminal ()`
Nuskaityti studentus iš terminalo.
- `void generateFile (int kiekStud, int kiekND, std::string fileName)`
Sugeneruoti testinių studentų failą.
- `void splitStudents (std::string dataFileName, std::string newFileName)`
Padalinti studentus į vargsiukus ir kietiakus pagal failo duomenis.
- `void testFileCreation (int kiekStud, int kiekND, const std::string &fileName)`
Matuoti failo generavimo laiką.
- `void testDataProcessing (const std::string &fileName)`
Matuoti duomenų nuskaitymo ir skirstymo laiką.
- `void testTime (int testSize, int ndSize)`
Paleidžia laiko testus.
- `void splitVector1 (const std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai, std::vector< Studentas > &kietiakai)`
Vector skirstymas kuriant du naujus konteinerius.
- `void splitVector2 (std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai)`
Vector skirstymas paliekant kietiakus pradiniam konteineriui.
- `void splitVector3 (std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai)`
Vector skirstymas naudojant std::partition.
- `void splitList1 (const std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai, std::list< Studentas > &kietiakai)`
List skirstymas kuriant du naujus konteinerius.
- `void splitList2 (std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai)`
List skirstymas naudojant erase.
- `void splitList3 (std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai)`
List skirstymas naudojant splice.
- `void splitDeque1 (const std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai, std::deque< Studentas > &kietiakai)`
Deque skirstymas kuriant du naujus konteinerius.
- `void splitDeque2 (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai)`
Deque skirstymas naudojant erase.
- `void splitDeque3 (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai)`
Deque skirstymas naudojant std::partition.

8.4.1 Funkcijos Dokumentacija

8.4.1.1 generateFile()

```
void generateFile (
    int kiekStud,
    int kiekND,
    std::string fileName)
```

Sugeneruoja testiniu studentu faila.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 388.

8.4.1.2 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s)
```

Isveda viena studenta i srauta.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 196.

8.4.1.3 operator>>()

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s)
```

Nuskaito viena studenta is srauto.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 144.

8.4.1.4 readFile()

```
std::vector< Studentas > readFile (
    const std::string & filename,
    bool saveND = false)
```

Nuskaito studentus is failo.

Parametrai

<i>filename</i>	Failo pavadinimas.
<i>saveND</i>	Ar saugoti ND pazymius objekte.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 207.

8.4.1.5 readTerminal()

```
std::vector< Studentas > readTerminal ()
```

Nuskaito studentus is terminalo.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 239.

8.4.1.6 splitDeque1()

```
void splitDeque1 (  
    const std::deque< Studentas > & studentai,  
    std::deque< Studentas > & vargsiukai,  
    std::deque< Studentas > & kietiakai)
```

Deque skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 582.

8.4.1.7 splitDeque2()

```
void splitDeque2 (  
    std::deque< Studentas > & studentai,  
    std::deque< Studentas > & vargsiukai)
```

Deque skirstymas naudojant erase.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 594.

8.4.1.8 splitDeque3()

```
void splitDeque3 (  
    std::deque< Studentas > & studentai,  
    std::deque< Studentas > & vargsiukai)
```

Deque skirstymas naudojant std::partition.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 606.

8.4.1.9 splitList1()

```
void splitList1 (  
    const std::list< Studentas > & studentai,  
    std::list< Studentas > & vargsiukai,  
    std::list< Studentas > & kietiakai)
```

List skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 545.

8.4.1.10 splitList2()

```
void splitList2 (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsiukai)
```

List skirstymas naudojant erase.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 557.

8.4.1.11 splitList3()

```
void splitList3 (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsiukai)
```

List skirstymas naudojant splice.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 569.

8.4.1.12 splitStudents()

```
void splitStudents (
    std::string dataFileName,
    std::string newFileName)
```

Padalina studentus i vargsiukus ir kietiokus pagal failo duomenis.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 421.

8.4.1.13 splitVector1()

```
void splitVector1 (
    const std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai,
    std::vector< Studentas > & kietiakai)
```

Vector skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 506.

8.4.1.14 splitVector2()

```
void splitVector2 (
    std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai)
```

Vector skirstymas paliekant kietiokus pradiniam konteineriye.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 518.

8.4.1.15 splitVector3()

```
void splitVector3 (
    std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai)
```

Vector skirstymas naudojant `std::partition`.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 530.

8.4.1.16 testDataProcessing()

```
void testDataProcessing (
    const std::string & fileName)
```

Matuoja duomenų nuskaitymo ir skirstymo laiką.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 476.

8.4.1.17 testFileCreation()

```
void testFileCreation (
    int kiekStud,
    int kiekND,
    const std::string & fileName)
```

Matuoja failo generavimo laiką.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 467.

8.4.1.18 testTime()

```
void testTime (
    int testSize,
    int ndSize)
```

Paleidžia laiko testus.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 499.

8.5 funkcijos.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "funkcijos.h"
00002
00003 // Zmogus
00004 Zmogus::Zmogus()
00005     : vardas_("", pavarde_("")) {}
00006
00007 Zmogus::Zmogus(const std::string& vardas, const std::string& pavarde)
00008     : vardas_(vardas), pavarde_(pavarde) {}
00009
00010 const std::string& Zmogus::getVardas() const {
00011     return vardas_;
00012 }
00013
00014 const std::string& Zmogus::getPavarde() const {
00015     return pavarde_;
00016 }
00017
00018 void Zmogus::setVardas(const std::string& value) {
00019     vardas_ = value;
00020 }
00021
00022 void Zmogus::setPavarde(const std::string& value) {
00023     pavarde_ = value;
00024 }
00025
00026 // Studentas: konstruktoriai
00027 Studentas::Studentas()
00028     : Zmogus(),
00029       namuDarbaiVid100_(0),
00030       namuDarbaiMed100_(0),
00031       egzaminas_(0),
00032       ND_() {}
00033
00034 Studentas::Studentas(const std::string& vardas,
00035                      const std::string& pavarde,
00036                      uint8_t egzaminas,
00037                      uint16_t namuDarbaiVid100,
00038                      uint16_t namuDarbaiMed100,
00039                      const std::vector<short int>& ND)
00040     : Zmogus(vardas, pavarde),
00041       namuDarbaiVid100_(namuDarbaiVid100),
00042       namuDarbaiMed100_(namuDarbaiMed100),
00043       egzaminas_(egzaminas),
00044       ND_(ND) {}
00045
00046 Studentas::~Studentas() = default;
00047
00048 Studentas::Studentas(const Studentas& other)
00049     : Zmogus(other),
00050       namuDarbaiVid100_(other.namuDarbaiVid100_),
00051       namuDarbaiMed100_(other.namuDarbaiMed100_),
00052       egzaminas_(other egzaminas_),
00053       ND_(other.ND_) {}
00054
00055 Studentas& Studentas::operator=(const Studentas& other) {
00056     if (this != &other) {
00057         vardas_ = other.vardas_;
00058         pavarde_ = other.pavarde_;
00059         namuDarbaiVid100_ = other.namuDarbaiVid100_;
00060         namuDarbaiMed100_ = other.namuDarbaiMed100_;
00061         egzaminas_ = other egzaminas_;
00062         ND_ = other.ND_;
00063     }
00064     return *this;
00065 }
00066
00067 Studentas& Studentas(Studentas&& other) noexcept
00068     : Zmogus(std::move(other.vardas_), std::move(other.pavarde_)),
00069       namuDarbaiVid100_(other.namuDarbaiVid100_),
00070       namuDarbaiMed100_(other.namuDarbaiMed100_),
00071       egzaminas_(other egzaminas_),
00072       ND_(std::move(other.ND_)) {
00073     other.namuDarbaiVid100_ = 0;
00074     other.namuDarbaiMed100_ = 0;
00075     other egzaminas_ = 0;
00076 }
00077
00078 Studentas& Studentas::operator=(Studentas&& other) noexcept {
00079     if (this != &other) {
00080         vardas_ = std::move(other.vardas_);
00081         pavarde_ = std::move(other.pavarde_);
00082         namuDarbaiVid100_ = other.namuDarbaiVid100_;
```

```

00083         namuDarbaiMed100_ = other.namuDarbaiMed100_;
00084         egzaminas_ = other egzaminas_;
00085         ND_ = std::move(other.ND_);
00086
00087         other.namuDarbaiVid100_ = 0;
00088         other.namuDarbaiMed100_ = 0;
00089         other egzaminas_ = 0;
00090     }
00091     return *this;
00092 }
00093
00094 // Studentas metodai
00095 uint16_t Studentas::getNamuDarbaiVid100() const {
00096     return namuDarbaiVid100_;
00097 }
00098
00099 uint16_t Studentas::getNamuDarbaiMed100() const {
00100     return namuDarbaiMed100_;
00101 }
00102
00103 uint8_t Studentas::getEgzaminas() const {
00104     return egzaminas_;
00105 }
00106
00107 const std::vector<short int>& Studentas::getND() const {
00108     return ND_;
00109 }
00110
00111 void Studentas::setNamuDarbaiVid100(uint16_t value) {
00112     namuDarbaiVid100_ = value;
00113 }
00114
00115 void Studentas::setNamuDarbaiMed100(uint16_t value) {
00116     namuDarbaiMed100_ = value;
00117 }
00118
00119 void Studentas::setEgzaminas(uint8_t value) {
00120     egzaminas_ = value;
00121 }
00122
00123 void Studentas::setND(const std::vector<short int>& value) {
00124     ND_ = value;
00125 }
00126
00127 void Studentas::addND(short int pazymys) {
00128     ND_.push_back(pazymys);
00129 }
00130
00131 double Studentas::galutinisVid() const {
00132     return (namuDarbaiVid100_ / 100.0) * 0.4 + egzaminas_ * 0.6;
00133 }
00134
00135 double Studentas::galutinisMed() const {
00136     return (namuDarbaiMed100_ / 100.0) * 0.4 + egzaminas_ * 0.6;
00137 }
00138
00139 std::string Studentas::tipas() const {
00140     return "Studentas";
00141 }
00142
00143 // Perdengti operatoriai
00144 std::istream& operator>(std::istream& in, Studentas& s) {
00145     std::string vardas, pavarde;
00146     in >> vardas >> pavarde;
00147
00148     if (!in) return in;
00149
00150     s.setVardas(vardas);
00151     s.setPavarde(pavarde);
00152
00153     std::vector<short int> nd;
00154     int x = 0;
00155
00156     while (in >> x) {
00157         nd.push_back(static_cast<short int>(x));
00158         if (in.peek() == '\n') break;
00159     }
00160
00161     if (!nd.empty()) {
00162         s.setEgzaminas(static_cast<uint8_t>(nd.back()));
00163         nd.pop_back();
00164
00165         int suma = 0;
00166         for (short int v : nd) {
00167             suma += v;
00168         }
00169     }

```

```

00170         std::sort(nd.begin(), nd.end());
00171
00172         const std::size_t n = nd.size();
00173         double med = 0.0;
00174
00175         if (n > 0) {
00176             if (n % 2 == 0) {
00177                 med = (nd[n / 2 - 1] + nd[n / 2]) / 2.0;
00178             } else {
00179                 med = nd[n / 2];
00180             }
00181         }
00182
00183         s.setNamuDarbaiVid100(
00184             (n > 0)
00185             ? static_cast<uint16_t>((static_cast<double>(suma) / static_cast<double>(n)) * 100.0)
00186             : 0
00187         );
00188
00189         s.setNamuDarbaiMed100(static_cast<uint16_t>(med * 100.0));
00190         s.setND(nd);
00191     }
00192
00193     return in;
00194 }
00195
00196 std::ostream& operator<<(std::ostream& out, const Studentas& s) {
00197     out << std::left
00198         << std::setw(12) << s.getVardas()
00199         << std::setw(15) << s.getPavarde()
00200         << std::setw(10) << s.galutinisVid()
00201         << std::setw(10) << s.galutinisMed();
00202
00203     return out;
00204 }
00205
00206 // Failo skaitymas
00207 std::vector<Studentas> readFile(const std::string& filename, bool saveND) {
00208     std::ifstream in(filename);
00209     std::vector<Studentas> studentai;
00210
00211     if (!in) {
00212         std::cout << "Nepavyko atidaryti failo\n";
00213         return studentai;
00214     }
00215
00216     std::string line;
00217     std::getline(in, line); // header
00218
00219     while (std::getline(in, line)) {
00220         if (line.empty()) continue;
00221
00222         std::stringstream ss(line);
00223         Studentas temp;
00224         ss >> temp;
00225
00226         if (temp.getVardas().empty() || temp.getPavarde().empty()) continue;
00227
00228         if (!saveND) {
00229             temp.setND({});
00230         }
00231
00232         studentai.push_back(temp);
00233     }
00234
00235     return studentai;
00236 }
00237
00238 // Skaitymas is terminalo
00239 std::vector<Studentas> readTerminal() {
00240     std::vector<Studentas> studentai;
00241     Studentas temp;
00242
00243     int mode = 0;
00244     int kiekND = 0;
00245     int mokiniai = 0;
00246
00247     std::cout << "Rezimas:\n1 - rankinis\n2 - random pazymiai\n3 - random vardai ir pazymiai\n";
00248     char whatMode[2]{};
00249     std::cin.getline(whatMode, 2);
00250
00251     switch (whatMode[0]) {
00252     case '1':
00253         mode = 1;
00254         break;
00255     case '2': {

```

```

00257         mode = 2;
00258         std::cout << "Namu darbu pazymiu skaicius: ";
00259         std::string ndSkaicius;
00260         std::getline(std::cin, ndSkaicius);
00261         try {
00262             kiekND = std::stoi(ndSkaicius);
00263         } catch (...) {
00264             std::cout << "Ne skaicius!\n";
00265             return {};
00266         }
00267         break;
00268     }
00269
00270     case '3': {
00271         mode = 3;
00272         std::cout << "Namu darbu pazymiu skaicius:\n";
00273         try {
00274             std::cin >> kiekND;
00275             std::cout << "Mokiniu kiekis:\n";
00276             std::cin >> mokiniai;
00277             std::cin.ignore();
00278         } catch (...) {
00279             std::cout << "Ne skaicius!\n";
00280             return {};
00281         }
00282         break;
00283     }
00284
00285     default:
00286         std::cout << "Netinkamas\n";
00287         return {};
00288 }
00289
00290 while (!(mode == 3 && mokiniai == 0)) {
00291     temp = Studentas{};
00292     std::string stringTemp;
00293
00294     std::random_device rd;
00295     std::mt19937 gen(rd());
00296     std::uniform_int_distribution<> distrib(0, 10);
00297
00298     if (mode != 3) {
00299         std::cout << "Norint baigti parasykite 'baigti'\nVardas: ";
00300         std::getline(std::cin, stringTemp);
00301         if (stringTemp == "baigti") break;
00302         temp.setVardas(stringTemp);
00303
00304         std::cout << "Pavarde: ";
00305         std::getline(std::cin, stringTemp);
00306         temp.setPavarde(stringTemp);
00307     } else {
00308         std::string vardai[11] = {
00309             "Jonas", "Mantas", "Lukas", "Tomas", "Dovydas",
00310             "Karolis", "Rokas", "Paulius", "Andrius", "Gabrielius", "Dominykas"
00311         };
00312         std::string pavardes[11] = {
00313             "Kazlauskas", "Jankauskas", "Petrauskas", "Balciunas", "Zukauskas",
00314             "Vasiliauskas", "Butkus", "Navickas", "Urbonas", "Kavaliauskas", "Stankevicius"
00315         };
00316
00317         temp.setVardas(vardai[distrib(gen)]);
00318         temp.setPavarde(pavardes[distrib(gen)]);
00319     }
00320
00321     int suma = 0;
00322     int kiek = 0;
00323     std::vector<int> nd_pazymiai;
00324
00325     if (mode == 1) {
00326         std::cout << "Iveskite namu darbu pazymius (atskirtus tarpais): ";
00327         std::getline(std::cin, stringTemp);
00328
00329         std::stringstream ss(stringTemp);
00330         int paz = 0;
00331
00332         while (ss >> paz) {
00333             if (paz < 0 || paz > 10) {
00334                 std::cout << "Netinkamas pazymys (0-10). Ignoruojamas.\n";
00335                 continue;
00336             }
00337             suma += paz;
00338             nd_pazymiai.push_back(paz);
00339             kiek++;
00340         }
00341     } else {
00342         for (int i = 0; i < kiekND; i++) {
00343             int paz = distrib(gen);

```

```

00344         suma += paz;
00345         nd_pazymiai.push_back(paz);
00346         kiek++;
00347     }
00348
00349     temp.setEgzaminas(static_cast<uint8_t>(distrib(gen)));
00350 }
00351
00352 std::sort(nd_pazymiai.begin(), nd_pazymiai.end());
00353
00354 double nd_med = 0.0;
00355
00356 if (kiek > 0) {
00357     if (kiek % 2 == 0) {
00358         nd_med = (nd_pazymiai[kiek / 2 - 1] + nd_pazymiai[kiek / 2]) / 2.0;
00359     } else {
00360         nd_med = nd_pazymiai[kiek / 2];
00361     }
00362 }
00363
00364 temp.setNamuDarbaiMed100(static_cast<uint16_t>(nd_med * 100.0));
00365
00366 if (kiek > 0) {
00367     temp.setNamuDarbaiVid100(
00368         static_cast<uint16_t>((static_cast<double>(suma) / static_cast<double>(kiek)) * 100.0)
00369     );
00370 } else {
00371     temp.setNamuDarbaiVid100(0);
00372 }
00373
00374 if (mode == 1) {
00375     std::cout << "Egzamino pazymys: ";
00376     std::getline(std::cin, stringTemp);
00377     temp.setEgzaminas(static_cast<uint8_t>(std::stoi(stringTemp)));
00378 }
00379
00380 studentai.push_back(temp);
00381 mokiniai--;
00382 }
00383
00384 return studentai;
00385 }
00386
00387 // Failo generavimas
00388 void generateFile(int kiekStud, int kiekND, std::string fileName) {
00389     std::random_device rd;
00390     std::mt19937 gen(rd());
00391     std::uniform_int_distribution<> distrib(0, 10);
00392
00393     std::ofstream outFile(fileName);
00394     if (!outFile) {
00395         std::cout << "Nepavyko sukurti failo.\n";
00396         return;
00397     }
00398
00399     outFile << std::left
00400         << std::setw(14) << "Vardas"
00401         << std::setw(17) << "Pavarde";
00402
00403     for (int i = 1; i <= kiekND; i++) {
00404         outFile << std::setw(10) << ("ND" + std::to_string(i));
00405     }
00406     outFile << std::setw(10) << "Egz." << "\n";
00407
00408     for (int i = 1; i <= kiekStud; i++) {
00409         outFile << std::left
00410             << std::setw(14) << ("Vardas" + std::to_string(i))
00411             << std::setw(17) << ("Pavarde" + std::to_string(i));
00412
00413         for (int j = 0; j <= kiekND; j++) {
00414             outFile << std::setw(10) << distrib(gen);
00415         }
00416         outFile << "\n";
00417     }
00418 }
00419
00420 // Studentu skaidymas i failus
00421 void splitStudents(std::string dataFileName, std::string newFileName) {
00422     std::vector<Studentas> mokiniai = readFile(dataFileName, true);
00423
00424     std::ofstream outGood("good_" + newFileName);
00425     std::ofstream outBad("bad_" + newFileName);
00426
00427     if (!outGood || !outBad) {
00428         std::cout << "Nepavyko sukurti failo.\n";
00429         return;
00430     }
00431 }

```

```

00431
00432     if (mokiniai.empty()) return;
00433
00434     int kiekND = static_cast<int>(mokiniai[0].getND().size());
00435
00436     auto printHeader = [&](std::ostream& out) {
00437         out << std::left
00438             << std::setw(14) << "Vardas"
00439             << std::setw(17) << "Pavarde";
00440
00441         for (int i = 1; i <= kiekND; i++) {
00442             out << std::setw(10) << ("ND" + std::to_string(i));
00443         }
00444
00445         out << std::setw(10) << "Egz." << "\n";
00446     };
00447
00448     printHeader(outGood);
00449     printHeader(outBad);
00450
00451     for (const auto& s : mokiniai) {
00452         std::ostream& out = (s.galutinisVid() >= 5.0) ? outGood : outBad;
00453
00454         out << std::left
00455             << std::setw(14) << s.getVardas()
00456             << std::setw(17) << s.getPavarde();
00457
00458         for (short int nd : s.getND()) {
00459             out << std::setw(10) << nd;
00460         }
00461
00462         out << std::setw(10) << static_cast<int>(s.getEgzaminas()) << "\n";
00463     }
00464 }
00465
00466 // Testavimo funkcijos
00467 void testFileCreation(int kiekStud, int kiekND, const std::string& fileName) {
00468     auto start = std::chrono::high_resolution_clock::now();
00469     generateFile(kiekStud, kiekND, fileName);
00470     auto end = std::chrono::high_resolution_clock::now();
00471
00472     std::cout << "Failo sukurimo laikas: "
00473               << std::chrono::duration<double>(end - start).count() << " s\n";
00474 }
00475
00476 void testDataProcessing(const std::string& fileName) {
00477     auto t0 = std::chrono::high_resolution_clock::now();
00478     std::vector<Studentas> students = readFile(fileName, true);
00479     auto t1 = std::chrono::high_resolution_clock::now();
00480
00481     std::vector<Studentas> good, bad;
00482     for (const auto& s : students) {
00483         if (s.galutinisVid() >= 5.0) {
00484             good.push_back(s);
00485         } else {
00486             bad.push_back(s);
00487         }
00488     }
00489     auto t2 = std::chrono::high_resolution_clock::now();
00490
00491     std::cout << "Failo nuskaitymo laikas: "
00492               << std::chrono::duration<double>(t1 - t0).count() << " s\n";
00493     std::cout << "Studentu skirstymo laikas: "
00494               << std::chrono::duration<double>(t2 - t1).count() << " s\n";
00495     std::cout << "Bendras laikas: "
00496               << std::chrono::duration<double>(t2 - t0).count() << " s\n";
00497 }
00498
00499 void testTime(int testSize, int ndSize) {
00500     std::cout << "Dydis: " << testSize << "\n";
00501     testFileCreation(testSize, ndSize, "students.txt");
00502     testDataProcessing("students.txt");
00503 }
00504
00505 // VECTOR strategijos
00506 void splitVector1(const std::vector<Studentas>& studentai,
00507                  std::vector<Studentas>& vargsiukai,
00508                  std::vector<Studentas>& kietiakai) {
00509     for (const auto& s : studentai) {
00510         if (s.galutinisVid() < 5.0) {
00511             vargsiukai.push_back(s);
00512         } else {
00513             kietiakai.push_back(s);
00514         }
00515     }
00516 }
00517

```

```

00518 void splitVector2(std::vector<Studentas>& studentai,
00519                   std::vector<Studentas>& vargsiukai) {
00520     for (auto it = studentai.begin(); it != studentai.end(); ) {
00521         if (it->galutinisVid() < 5.0) {
00522             vargsiukai.push_back(*it);
00523             it = studentai.erase(it);
00524         } else {
00525             ++it;
00526         }
00527     }
00528 }
00529
00530 void splitVector3(std::vector<Studentas>& studentai,
00531                  std::vector<Studentas>& vargsiukai) {
00532     auto it = std::partition(studentai.begin(), studentai.end(),
00533                             [](const Studentas& s) {
00534                                 return s.galutinisVid() >= 5.0;
00535                             });
00536
00537     for (auto i = it; i != studentai.end(); ++i) {
00538         vargsiukai.push_back(*i);
00539     }
00540
00541     studentai.erase(it, studentai.end());
00542 }
00543
00544 // LIST strategijos
00545 void splitList1(const std::list<Studentas>& studentai,
00546                std::list<Studentas>& vargsiukai,
00547                std::list<Studentas>& kietiakai) {
00548     for (const auto& s : studentai) {
00549         if (s.galutinisVid() < 5.0) {
00550             vargsiukai.push_back(s);
00551         } else {
00552             kietiakai.push_back(s);
00553         }
00554     }
00555 }
00556
00557 void splitList2(std::list<Studentas>& studentai,
00558                std::list<Studentas>& vargsiukai) {
00559     for (auto it = studentai.begin(); it != studentai.end(); ) {
00560         if (it->galutinisVid() < 5.0) {
00561             vargsiukai.push_back(*it);
00562             it = studentai.erase(it);
00563         } else {
00564             ++it;
00565         }
00566     }
00567 }
00568
00569 void splitList3(std::list<Studentas>& studentai,
00570                std::list<Studentas>& vargsiukai) {
00571     for (auto it = studentai.begin(); it != studentai.end(); ) {
00572         if (it->galutinisVid() < 5.0) {
00573             auto temp = it++;
00574             vargsiukai.splice(vargsiukai.end(), studentai, temp);
00575         } else {
00576             ++it;
00577         }
00578     }
00579 }
00580
00581 // DEQUE strategijos
00582 void splitDeque1(const std::deque<Studentas>& studentai,
00583                  std::deque<Studentas>& vargsiukai,
00584                  std::deque<Studentas>& kietiakai) {
00585     for (const auto& s : studentai) {
00586         if (s.galutinisVid() < 5.0) {
00587             vargsiukai.push_back(s);
00588         } else {
00589             kietiakai.push_back(s);
00590         }
00591     }
00592 }
00593
00594 void splitDeque2(std::deque<Studentas>& studentai,
00595                  std::deque<Studentas>& vargsiukai) {
00596     for (auto it = studentai.begin(); it != studentai.end(); ) {
00597         if (it->galutinisVid() < 5.0) {
00598             vargsiukai.push_back(*it);
00599             it = studentai.erase(it);
00600         } else {
00601             ++it;
00602         }
00603     }
00604 }

```

```

00605
00606 void splitDeque3(std::deque<Studentas>& studentai,
00607                  std::deque<Studentas>& vargsiukai) {
00608     auto it = std::partition(studentai.begin(), studentai.end(),
00609                             [](const Studentas& s) {
00610                                 return s.galutinisVid() >= 5.0;
00611                             });
00612
00613     for (auto i = it; i != studentai.end(); ++i) {
00614         vargsiukai.push_back(*i);
00615     }
00616
00617     studentai.erase(it, studentai.end());
00618 }

```

8.6 funkcijos.h Failo Nuoroda

Studentu pažymiu sistemos klasiu ir funkciju deklaracijos.

```

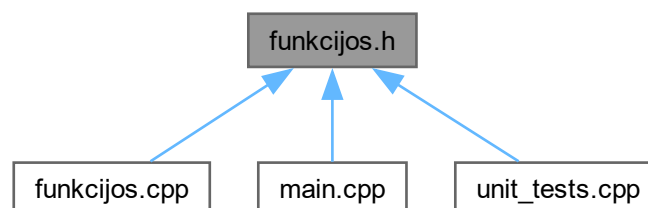
#include <algorithm>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <fstream>
#include <vector>
#include <list>
#include <deque>
#include <chrono>
#include <string>
#include <random>
#include <cstring>
#include <sstream>
#include <utility>

```

Įtraukimo priklausomybių diagrama funkcijos.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Zmogus](#)
Abstrakti bazinė klase bendriems zmogaus duomenims saugoti.
- class [Studentas](#)
Studentas su namu darbu, egzamino ir galutinio balo duomenimis.

Funkcijos

- `std::istream & operator>> (std::istream &in, Studentas &s)`
Nuskaityti viena studenta iš srauto.
- `std::ostream & operator<< (std::ostream &out, const Studentas &s)`
Isveda viena studenta į srautą.
- `std::vector< Studentas > readFile (const std::string &filename, bool saveND=false)`
Nuskaityti studentus iš failo.
- `std::vector< Studentas > readTerminal ()`
Nuskaityti studentus iš terminalo.
- `void generateFile (int kiekStud, int kiekND, std::string fileName)`
Sugeneruoja testiniu studentu failą.
- `void splitStudents (std::string dataFileName, std::string newFileName)`
Padalina studentus į vargsiukus ir kietiakus pagal failo duomenis.
- `void testFileCreation (int kiekStud, int kiekND, const std::string &fileName)`
Matuoja failo generavimo laiką.
- `void testDataProcessing (const std::string &fileName)`
Matuoja duomenų nuskaitymo ir skirstymo laiką.
- `void testTime (int testSize, int ndSize)`
Paleidžia laiko testus.
- `void testClass ()`
Paleidžia senesnius klases testus, pasiekiamus per programos meniu.
- `void splitVector1 (const std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai, std::vector< Studentas > &kietiakai)`
Vector skirstymas kuriant du naujus konteinerius.
- `void splitVector2 (std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai)`
Vector skirstymas paliekant kietiakus pradiniam konteineriui.
- `void splitVector3 (std::vector< Studentas > &studentai, std::vector< Studentas > &vargsiukai)`
Vector skirstymas naudojant std::partition.
- `void splitList1 (const std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai, std::list< Studentas > &kietiakai)`
List skirstymas kuriant du naujus konteinerius.
- `void splitList2 (std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai)`
List skirstymas naudojant erase.
- `void splitList3 (std::list< Studentas > &studentai, std::list< Studentas > &vargsiukai)`
List skirstymas naudojant splice.
- `void splitDeque1 (const std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai, std::deque< Studentas > &kietiakai)`
Deque skirstymas kuriant du naujus konteinerius.
- `void splitDeque2 (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai)`
Deque skirstymas naudojant erase.
- `void splitDeque3 (std::deque< Studentas > &studentai, std::deque< Studentas > &vargsiukai)`
Deque skirstymas naudojant std::partition.

8.6.1 Smulkus aprašymas

Studentu pazymiu sistemos klasiu ir funkciju deklaracijos.

Faile deklaruojama abstrakti klase [Zmogus](#), is jos paveldima klase [Studentas](#), ivesties/isvesties operatoriai, failu apdorojimo funkcijos ir kontaineriu skirstymo strategijos.

Apibrėžimas faile [funkcijos.h](#).

8.6.2 Funkcijos Dokumentacija

8.6.2.1 generateFile()

```
void generateFile (
    int kiekStud,
    int kiekND,
    std::string fileName)
```

Sugeneruoja testiniu studentu faila.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 388.

8.6.2.2 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & s)
```

Isveda viena studenta i srauta.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 196.

8.6.2.3 operator>>()

```
std::istream & operator>> (
    std::istream & in,
    Studentas & s)
```

Nuskaito viena studenta is srauto.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 144.

8.6.2.4 readFile()

```
std::vector< Studentas > readFile (
    const std::string & filename,
    bool saveND = false)
```

Nuskaityti studentus iš failo.

Parametrai

<i>filename</i>	Failo pavadinimas.
<i>saveND</i>	Ar saugoti ND pazymius objekte.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 207.

8.6.2.5 readTerminal()

```
std::vector< Studentas > readTerminal ()
```

Nuskaityti studentus iš terminalo.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 239.

8.6.2.6 splitDeque1()

```
void splitDeque1 (
    const std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsiukai,
    std::deque< Studentas > & kietiakai)
```

Deque skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 582.

8.6.2.7 splitDeque2()

```
void splitDeque2 (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsiukai)
```

Deque skirstymas naudojant erase.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 594.

8.6.2.8 splitDeque3()

```
void splitDeque3 (
    std::deque< Studentas > & studentai,
    std::deque< Studentas > & vargsiukai)
```

Deque skirstymas naudojant std::partition.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 606.

8.6.2.9 splitList1()

```
void splitList1 (
    const std::list< Studentas > & studentai,
    std::list< Studentas > & vargsiukai,
    std::list< Studentas > & kietiakai)
```

List skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 545.

8.6.2.10 splitList2()

```
void splitList2 (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsiukai)
```

List skirstymas naudojant erase.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 557.

8.6.2.11 splitList3()

```
void splitList3 (
    std::list< Studentas > & studentai,
    std::list< Studentas > & vargsiukai)
```

List skirstymas naudojant splice.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 569.

8.6.2.12 splitStudents()

```
void splitStudents (
    std::string dataFileName,
    std::string newFileName)
```

Padalina studentus i vargsiukus ir kietiakus pagal failo duomenis.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 421.

8.6.2.13 splitVector1()

```
void splitVector1 (
    const std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai,
    std::vector< Studentas > & kietiakai)
```

Vector skirstymas kuriant du naujus konteinerius.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 506.

8.6.2.14 splitVector2()

```
void splitVector2 (
    std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai)
```

Vector skirstymas paliekant kietiakus pradiniam konteineriui.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 518.

8.6.2.15 splitVector3()

```
void splitVector3 (
    std::vector< Studentas > & studentai,
    std::vector< Studentas > & vargsiukai)
```

Vector skirstymas naudojant `std::partition`.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 530.

8.6.2.16 testClass()

```
void testClass ()
```

Paleidžia senesnius klases testus, pasiekiamus per programos meniu.

8.6.2.17 testDataProcessing()

```
void testDataProcessing (
    const std::string & fileName)
```

Matuoja duomenų nuskaitymo ir skirstymo laiką.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 476.

8.6.2.18 testFileCreation()

```
void testFileCreation (
    int kiekStud,
    int kiekND,
    const std::string & fileName)
```

Matuoja failo generavimo laiką.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 467.

8.6.2.19 testTime()

```
void testTime (
    int testSize,
    int ndSize)
```

Paleidzia laiko testus.

Apibrėžimas failo [funkcijos.cpp](#) eilutėje 499.

8.7 funkcijos.h

Eiti į šio failo dokumentaciją.

```
00001
00009 #ifndef OOP_PAZYMIU_SISTEMA_FUNKCIJOS_H
00010 #define OOP_PAZYMIU_SISTEMA_FUNKCIJOS_H
00011
00012 #include <algorithm>
00013 #include <cstdint>
00014 #include <iomanip>
00015 #include <iostream>
00016 #include <fstream>
00017 #include <vector>
00018 #include <list>
00019 #include <deque>
00020 #include <chrono>
00021 #include <string>
00022 #include <random>
00023 #include <cstring>
00024 #include <sstream>
00025 #include <utility>
00026
00033 class Zmogus {
00034 protected:
00035     std::string vardas_;
00036     std::string pavarde_;
00037
00038 public:
00040     Zmogus();
00041
00047     Zmogus(const std::string& vardas, const std::string& pavarde);
00048
00050     virtual ~Zmogus() = default;
00051
00053     const std::string& getVardas() const;
00054
00056     const std::string& getPavarde() const;
00057
00059     void setVardas(const std::string& value);
00060
00062     void setPavarde(const std::string& value);
00063
00065     virtual std::string tipas() const = 0;
00066 };
00067
00074 class Studentas : public Zmogus {
00075 private:
00076     uint16_t namuDarbaiVid100_;
00077     uint16_t namuDarbaiMed100_;
00078     uint8_t egzaminas_;
00079     std::vector<short int> ND_;
00080
00081 public:
00083     Studentas();
00084
00086     ~Studentas();
00087
00097     Studentas(const std::string& vardas,
00098               const std::string& pavarde,
00099               uint8_t egzaminas,
00100               uint16_t namuDarbaiVid100,
00101               uint16_t namuDarbaiMed100,
00102               const std::vector<short int>& ND = {});
00103
00105     Studentas(const Studentas& other);
00106
```

```

00108     Studentas& operator=(const Studentas& other);
00109
00111     Studentas(Studentas&& other) noexcept;
00112
00114     Studentas& operator=(Studentas&& other) noexcept;
00115
00117     uint16_t getNamuDarbaiVid100() const;
00118
00120     uint16_t getNamuDarbaiMed100() const;
00121
00123     uint8_t getEgzaminas() const;
00124
00126     const std::vector<short int>& getND() const;
00127
00129     void setNamuDarbaiVid100(uint16_t value);
00130
00132     void setNamuDarbaiMed100(uint16_t value);
00133
00135     void setEgzaminas(uint8_t value);
00136
00138     void setND(const std::vector<short int>& value);
00139
00141     void addND(short int pazymys);
00142
00144     double galutinisVid() const;
00145
00147     double galutinisMed() const;
00148
00150     std::string tipas() const override;
00151 };
00152
00154 std::istream& operator>(std::istream& in, Studentas& s);
00155
00157 std::ostream& operator<(std::ostream& out, const Studentas& s);
00158
00160 std::vector<Studentas> readFile(const std::string& filename, bool saveND = false);
00161
00163 std::vector<Studentas> readTerminal();
00164
00166 void generateFile(int kiekStud, int kiekND, std::string fileName);
00167
00169 void splitStudents(std::string dataFileName, std::string newFileName);
00170
00172 void testFileCreation(int kiekStud, int kiekND, const std::string& fileName);
00173
00175 void testDataProcessing(const std::string& fileName);
00176
00178 void testTime(int testSize, int ndSize);
00179
00181 void testClass();
00182
00184 void splitVector1(const std::vector<Studentas>& studentai,
00185                  std::vector<Studentas>& vargsiukai,
00186                  std::vector<Studentas>& kietiakai);
00187
00189 void splitVector2(std::vector<Studentas>& studentai,
00190                  std::vector<Studentas>& vargsiukai);
00191
00193 void splitVector3(std::vector<Studentas>& studentai,
00194                  std::vector<Studentas>& vargsiukai);
00195
00197 void splitList1(const std::list<Studentas>& studentai,
00198                std::list<Studentas>& vargsiukai,
00199                std::list<Studentas>& kietiakai);
00200
00202 void splitList2(std::list<Studentas>& studentai,
00203                std::list<Studentas>& vargsiukai);
00204
00206 void splitList3(std::list<Studentas>& studentai,
00207                std::list<Studentas>& vargsiukai);
00208
00210 void splitDeque1(const std::deque<Studentas>& studentai,
00211                 std::deque<Studentas>& vargsiukai,
00212                 std::deque<Studentas>& kietiakai);
00213
00215 void splitDeque2(std::deque<Studentas>& studentai,
00216                 std::deque<Studentas>& vargsiukai);
00217
00219 void splitDeque3(std::deque<Studentas>& studentai,
00220                 std::deque<Studentas>& vargsiukai);
00221
00222 #endif

```

8.8 main.cpp Failo Nuoroda

```
#include "funkcijos.h"
```

Įtraukimo priklausomybių diagrama main.cpp:



Funkcijos

- int [main](#) ()

8.8.1 Funkcijos Dokumentacija

8.8.1.1 main()

```
int main ()
```

Apibrėžimas failo [main.cpp](#) eilutėje 3.

8.9 main.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "funkcijos.h"
00002
00003 int main() {
00004     std::vector<Studentas> studentai;
00005
00006     std::cout << "Pasirinkite:\n";
00007     std::cout << "1 - mokiniu ivestis terminale\n";
00008     std::cout << "2 - mokiniu ivestis is failo\n";
00009     std::cout << "3 - generuoti faila\n";
00010     std::cout << "4 - padalinti mokinius i grupes is failo\n";
00011     std::cout << "5 - testuoti strategijas su vector\n";
00012     std::cout << "6 - testuoti strategijas su list\n";
00013     std::cout << "7 - testuoti strategijas su deque\n";
00014     std::cout << "8 - senas bendras laiko testas\n";
00015     std::cout << "9 - metodu testas\n";
00016
00017     char pasirinkimas;
00018     std::cin >> pasirinkimas;
00019     std::cin.ignore();
00020
00021     if (pasirinkimas == '1') {
00022         studentai = readTerminal();
00023     }
00024     else if (pasirinkimas == '2') {
00025         std::string failas;
00026         std::cout << "Iveskite failo pavadinima: ";
00027         std::getline(std::cin, failas);
00028
00029         studentai = readFile(failas);
00030     }
00031     else if (pasirinkimas == '3') {
00032         int kiekStud, kiekND;

```



```

00033         std::string fileName;
00034
00035         std::cout << "Failo pavadinimas:\n";
00036         std::cin >> fileName;
00037
00038         std::cout << "Kiek studentu faile:\n";
00039         std::cin >> kiekStud;
00040
00041         std::cout << "Kiek namu darbu pazymiu:\n";
00042         std::cin >> kiekND;
00043
00044         generateFile(kiekStud, kiekND, fileName);
00045         return 0;
00046     }
00047     else if (pasirinkimas == '4') {
00048         std::string dataFileName, newFileName;
00049
00050         std::cout << "Mokiniu duomenu failo pavadinimas:\n";
00051         std::cin >> dataFileName;
00052
00053         std::cout << "Padalintu mokiniu failo pavadinimas:\n";
00054         std::cin >> newFileName;
00055
00056         splitStudents(dataFileName, newFileName);
00057         return 0;
00058     }
00059     else if (pasirinkimas == '5') {
00060         std::string failas;
00061         std::cout << "Iveskite failo pavadinima: ";
00062         std::getline(std::cin, failas);
00063
00064         std::vector<Studentas> studentaiTest = readFile(failas);
00065
00066         if (studentaiTest.empty()) {
00067             std::cout << "Nepavyko nuskaityti studentu.\n";
00068             return 0;
00069         }
00070
00071         std::cout << "\nPasirinkite strategija:\n";
00072         std::cout << "1 - kurti du naujus konteinerius\n";
00073         std::cout << "2 - kurti tik vargsiukus ir trinti is bendro\n";
00074         std::cout << "3 - naudoti partition\n";
00075
00076         char strategija;
00077         std::cin >> strategija;
00078
00079         auto start = std::chrono::high_resolution_clock::now();
00080
00081         if (strategija == '1') {
00082             std::vector<Studentas> vargsiukai;
00083             std::vector<Studentas> kietiakai;
00084             splitVector1(studentaiTest, vargsiukai, kietiakai);
00085
00086             auto end = std::chrono::high_resolution_clock::now();
00087             std::chrono::duration<double> elapsed = end - start;
00088
00089             std::cout << "\nVECTOR 1 strategija baigta\n";
00090             std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00091             std::cout << "Kietiakai: " << kietiakai.size() << "\n";
00092             std::cout << "Laikas: " << elapsed.count() << " s\n";
00093         }
00094         else if (strategija == '2') {
00095             std::vector<Studentas> vargsiukai;
00096             splitVector2(studentaiTest, vargsiukai);
00097
00098             auto end = std::chrono::high_resolution_clock::now();
00099             std::chrono::duration<double> elapsed = end - start;
00100
00101             std::cout << "\nVECTOR 2 strategija baigta\n";
00102             std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00103             std::cout << "Kietiakai: " << studentaiTest.size() << "\n";
00104             std::cout << "Laikas: " << elapsed.count() << " s\n";
00105         }
00106         else if (strategija == '3') {
00107             std::vector<Studentas> vargsiukai;
00108             splitVector3(studentaiTest, vargsiukai);
00109
00110             auto end = std::chrono::high_resolution_clock::now();
00111             std::chrono::duration<double> elapsed = end - start;
00112
00113             std::cout << "\nVECTOR 3 strategija baigta\n";
00114             std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00115             std::cout << "Kietiakai: " << studentaiTest.size() << "\n";
00116             std::cout << "Laikas: " << elapsed.count() << " s\n";
00117         }
00118         else {
00119             std::cout << "Netinkama strategija.\n";

```

```

00120     }
00121
00122     return 0;
00123 }
00124 else if (pasirinkimas == '6') {
00125     std::string failas;
00126     std::cout << "Iveskite failo pavadinima: ";
00127     std::getline(std::cin, failas);
00128
00129     std::vector<Studentas> temp = readFile(failas);
00130     if (temp.empty()) {
00131         std::cout << "Nepavyko nuskaityti studentu.\n";
00132         return 0;
00133     }
00134
00135     std::list<Studentas> studentaiList;
00136     for (const auto& s : temp) {
00137         studentaiList.push_back(s);
00138     }
00139
00140     std::cout << "\nPasirinkite strategija:\n";
00141     std::cout << "1 - kurti du naujus konteinerius\n";
00142     std::cout << "2 - kurti tik vargsiukus ir trinti is bendro\n";
00143     std::cout << "3 - naudoti splice\n";
00144
00145     char strategija;
00146     std::cin >> strategija;
00147
00148     auto start = std::chrono::high_resolution_clock::now();
00149
00150     if (strategija == '1') {
00151         std::list<Studentas> vargsiukai;
00152         std::list<Studentas> kietiakai;
00153         splitList1(studentaiList, vargsiukai, kietiakai);
00154
00155         auto end = std::chrono::high_resolution_clock::now();
00156         std::chrono::duration<double> elapsed = end - start;
00157
00158         std::cout << "\nLIST 1 strategija baigta\n";
00159         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00160         std::cout << "Kietiakai: " << kietiakai.size() << "\n";
00161         std::cout << "Laikas: " << elapsed.count() << " s\n";
00162     }
00163     else if (strategija == '2') {
00164         std::list<Studentas> vargsiukai;
00165         splitList2(studentaiList, vargsiukai);
00166
00167         auto end = std::chrono::high_resolution_clock::now();
00168         std::chrono::duration<double> elapsed = end - start;
00169
00170         std::cout << "\nLIST 2 strategija baigta\n";
00171         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00172         std::cout << "Kietiakai: " << studentaiList.size() << "\n";
00173         std::cout << "Laikas: " << elapsed.count() << " s\n";
00174     }
00175     else if (strategija == '3') {
00176         std::list<Studentas> vargsiukai;
00177         splitList3(studentaiList, vargsiukai);
00178
00179         auto end = std::chrono::high_resolution_clock::now();
00180         std::chrono::duration<double> elapsed = end - start;
00181
00182         std::cout << "\nLIST 3 strategija baigta\n";
00183         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00184         std::cout << "Kietiakai: " << studentaiList.size() << "\n";
00185         std::cout << "Laikas: " << elapsed.count() << " s\n";
00186     }
00187     else {
00188         std::cout << "Netinkama strategija.\n";
00189     }
00190
00191     return 0;
00192 }
00193 else if (pasirinkimas == '7') {
00194     std::string failas;
00195     std::cout << "Iveskite failo pavadinima: ";
00196     std::getline(std::cin, failas);
00197
00198     std::vector<Studentas> temp = readFile(failas);
00199     if (temp.empty()) {
00200         std::cout << "Nepavyko nuskaityti studentu.\n";
00201         return 0;
00202     }
00203
00204     std::deque<Studentas> studentaiDeque;
00205     for (const auto& s : temp) {
00206         studentaiDeque.push_back(s);

```

```

00207     }
00208
00209     std::cout << "\nPasirinkite strategija:\n";
00210     std::cout << "1 - kurti du naujus konteinerius\n";
00211     std::cout << "2 - kurti tik vargsiukus ir trinti is bendro\n";
00212     std::cout << "3 - naudoti partition\n";
00213
00214     char strategija;
00215     std::cin >> strategija;
00216
00217     auto start = std::chrono::high_resolution_clock::now();
00218
00219     if (strategija == '1') {
00220         std::deque<Studentas> vargsiukai;
00221         std::deque<Studentas> kietiakai;
00222         splitDeque1(studentaiDeque, vargsiukai, kietiakai);
00223
00224         auto end = std::chrono::high_resolution_clock::now();
00225         std::chrono::duration<double> elapsed = end - start;
00226
00227         std::cout << "\nDEQUEUE 1 strategija baigta\n";
00228         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00229         std::cout << "Kietiakai: " << kietiakai.size() << "\n";
00230         std::cout << "Laikas: " << elapsed.count() << " s\n";
00231     }
00232     else if (strategija == '2') {
00233         std::deque<Studentas> vargsiukai;
00234         splitDeque2(studentaiDeque, vargsiukai);
00235
00236         auto end = std::chrono::high_resolution_clock::now();
00237         std::chrono::duration<double> elapsed = end - start;
00238
00239         std::cout << "\nDEQUEUE 2 strategija baigta\n";
00240         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00241         std::cout << "Kietiakai: " << studentaiDeque.size() << "\n";
00242         std::cout << "Laikas: " << elapsed.count() << " s\n";
00243     }
00244     else if (strategija == '3') {
00245         std::deque<Studentas> vargsiukai;
00246         splitDeque3(studentaiDeque, vargsiukai);
00247
00248         auto end = std::chrono::high_resolution_clock::now();
00249         std::chrono::duration<double> elapsed = end - start;
00250
00251         std::cout << "\nDEQUEUE 3 strategija baigta\n";
00252         std::cout << "Vargsiukai: " << vargsiukai.size() << "\n";
00253         std::cout << "Kietiakai: " << studentaiDeque.size() << "\n";
00254         std::cout << "Laikas: " << elapsed.count() << " s\n";
00255     }
00256     else {
00257         std::cout << "Netinkama strategija.\n";
00258     }
00259
00260     return 0;
00261 }
00262 else if (pasirinkimas == '8') {
00263     int testSize, ndSize;
00264     std::cout << "Iveskite studentu kieki: ";
00265     std::cin >> testSize;
00266     std::cout << "Iveskite ND kieki: ";
00267     std::cin >> ndSize;
00268
00269     testTime(testSize, ndSize);
00270     return 0;
00271 }
00272 else if (pasirinkimas == '9') {
00273     testClass();
00274     return 0;
00275 }
00276 else {
00277     std::cout << "Netinkamas pasirinkimas\n";
00278     return 0;
00279 }
00280
00281 std::cout << "\nRikiavimas:\n";
00282 std::cout << "1 - Vardas\n";
00283 std::cout << "2 - Pavarde\n";
00284 std::cout << "3 - Galutinis (Vid.)\n";
00285 std::cout << "4 - Galutinis (Med.)\n";
00286
00287 char rikiavimas;
00288 std::cin >> rikiavimas;
00289
00290 auto start = std::chrono::high_resolution_clock::now();
00291
00292 switch (rikiavimas) {
00293     case '1':

```

```

00294         std::sort(studentai.begin(), studentai.end(),
00295             [](const Studentas& a, const Studentas& b) {
00296                 return a.getVardas() < b.getVardas();
00297             });
00298         break;
00299
00300     case '2':
00301         std::sort(studentai.begin(), studentai.end(),
00302             [](const Studentas& a, const Studentas& b) {
00303                 return a.getPavarde() < b.getPavarde();
00304             });
00305         break;
00306
00307     case '3':
00308         std::sort(studentai.begin(), studentai.end(),
00309             [](const Studentas& a, const Studentas& b) {
00310                 return a.galutinisVid() < b.galutinisVid();
00311             });
00312         break;
00313
00314     case '4':
00315         std::sort(studentai.begin(), studentai.end(),
00316             [](const Studentas& a, const Studentas& b) {
00317                 return a.galutinisMed() < b.galutinisMed();
00318             });
00319         break;
00320
00321     default:
00322         std::cout << "Neteisingas pasirinkimas, nerikiuojama.\n";
00323         break;
00324 }
00325
00326 auto end = std::chrono::high_resolution_clock::now();
00327 std::chrono::duration<double> elapsed = end - start;
00328
00329 char pasirinkimasOutput;
00330 std::cout << "Kur rodyti rezultatus? (1 - ekranas, 2 - failas): ";
00331 std::cin >> pasirinkimasOutput;
00332 std::cin.ignore();
00333
00334 std::ostream* out = &std::cout;
00335 std::ofstream outFile;
00336
00337 if (pasirinkimasOutput == '2') {
00338     std::string outputFileName;
00339     std::cout << "Iveskite failo pavadinima: ";
00340     std::getline(std::cin, outputFileName);
00341
00342     outFile.open(outputFileName);
00343     if (!outFile) {
00344         std::cout << "Nepavyko sukurti failo.\n";
00345         return 1;
00346     }
00347     out = &outFile;
00348 }
00349
00350 *out << std::left
00351     << std::setw(14) << "Vardas"
00352     << std::setw(17) << "Pavarde"
00353     << std::setw(17) << "Galutinis (Vid.)"
00354     << std::setw(17) << "Galutinis (Med.)"
00355     << "\n";
00356
00357 *out << std::string(65, '-') << "\n";
00358 *out << std::fixed << std::setprecision(2);
00359
00360 for (const auto& i : studentai) {
00361     *out << std::setw(14) << i.getVardas()
00362         << std::setw(17) << i.getPavarde()
00363         << std::setw(17) << i.galutinisVid()
00364         << std::setw(17) << i.galutinisMed() << "\n";
00365 }
00366
00367 if (!studentai.empty()) {
00368     double total = elapsed.count();
00369     double avg = total / static_cast<double>(studentai.size());
00370
00371     std::cout << "\n--- Laiko matavimas ---\n";
00372     std::cout << "Bendras laikas: " << total << " s\n";
00373     std::cout << "Studentu kiekis: " << studentai.size() << "\n";
00374     std::cout << "Vidutinis laikas/studentui: " << avg << " s\n";
00375 }
00376
00377 return 0;
00378 }

```